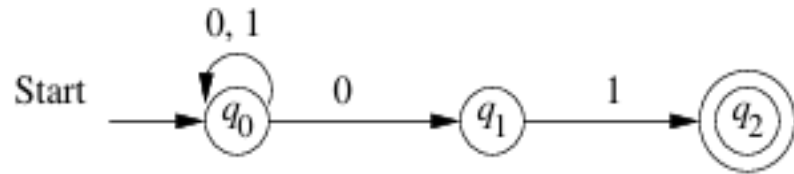


CSE 3204
Formal Language, Automata and Computability



Lecture 3
Equivalence of DFA and NFA

Continuation of Subset Construction



Transition diagram

	0	1
A	A	A
$\rightarrow B$	E	B
C	A	D
$*D$	A	A
E	E	F
$*F$	E	B
$*G$	A	D
$*H$	E	F

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$*\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$*\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

Figure 2.12: The complete subset construction from Fig. 2.9

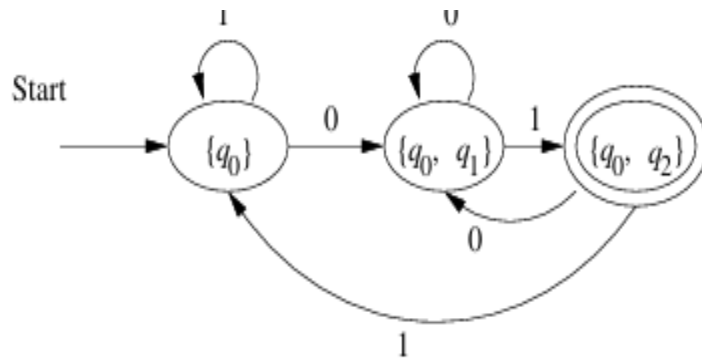
- Only three states B, E and F are reachable from B.
- Although the DFA generates 2^n states for an NFA with n states, many of the states in the DFA is not reachable from the start states.
- To avoid the exponential-time step of constructing transition table entries for every subset of states, we can perform “Lazy evaluation”.

Lazy Evaluation in Subset Construction

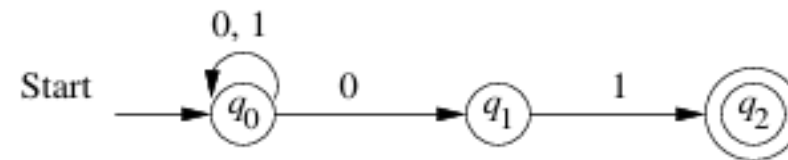
BASIS: We know for certain that the singleton set consisting only of N 's start state is accessible.

INDUCTION: Suppose we have determined that set S of states is accessible. Then for each input symbol a , compute the set of states $\delta_D(S, a)$; we know that these sets of states will also be accessible.

Final DFA:



Initial DFA:



Theorem 2.11: If $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ is the DFA constructed from NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ by the subset construction, then $L(D) = L(N)$.

PROOF: What we actually prove first, by induction on $|w|$, is that

$$\hat{\delta}_D(\{q_0\}, w) = \hat{\delta}_N(q_0, w)$$

BASIS: Let $|w| = 0$; that is, $w = \epsilon$. By the basis definitions of $\hat{\delta}$ for DFA's and NFA's, both $\hat{\delta}_D(\{q_0\}, \epsilon)$ and $\hat{\delta}_N(q_0, \epsilon)$ are $\{q_0\}$.

INDUCTION: Let w be of length $n + 1$, and assume the statement for length n . Break w up as $w = xa$, where a is the final symbol of w . By the inductive hypothesis, $\hat{\delta}_D(\{q_0\}, x) = \hat{\delta}_N(q_0, x)$. Let both these sets of N 's states be $\{p_1, p_2, \dots, p_k\}$.

The inductive part of the definition of $\hat{\delta}$ for NFA's tells us

$$\hat{\delta}_N(q_0, w) = \bigcup_{i=1}^k \delta_N(p_i, a) \quad (2.2)$$

The subset construction, on the other hand, tells us that

$$\delta_D(\{p_1, p_2, \dots, p_k\}, a) = \bigcup_{i=1}^k \delta_N(p_i, a) \quad (2.3)$$

Now, let us use (2.3) and the fact that $\hat{\delta}_D(\{q_0\}, x) = \{p_1, p_2, \dots, p_k\}$ in the inductive part of the definition of $\hat{\delta}$ for DFA's:

$$\hat{\delta}_D(\{q_0\}, w) = \delta_D(\hat{\delta}_D(\{q_0\}, x), a) = \delta_D(\{p_1, p_2, \dots, p_k\}, a) = \bigcup_{i=1}^n \delta_N(p_i, a) \quad (2.4)$$

Thus, Equations (2.2) and (2.4) demonstrate that $\hat{\delta}_D(\{q_0\}, w) = \hat{\delta}_N(q_0, w)$. When we observe that D and N both accept w if and only if $\hat{\delta}_D(\{q_0\}, w)$ or $\hat{\delta}_N(q_0, w)$, respectively, contain a state in F_N , we have a complete proof that $L(D) = L(N)$. $\square$

Theorem 2.12: A language L is accepted by some DFA if and only if L is accepted by some NFA.

PROOF: (If) The “if” part is the subset construction and Theorem 2.11.

(Only-if) This part is easy; we have only to convert a DFA into an identical NFA. Put intuitively, if we have the transition diagram for a DFA, we can also interpret it as the transition diagram of an NFA, which happens to have exactly one choice of transition in any situation. More formally, let $D = (Q, \Sigma, \delta_D, q_0, F)$ be a DFA. Define $N = (Q, \Sigma, \delta_N, q_0, F)$ to be the equivalent NFA, where δ_N is defined by the rule:

- If $\delta_D(q, a) = p$, then $\delta_N(q, a) = \{p\}$.

It is then easy to show by induction on $|w|$, that if $\hat{\delta}_D(q_0, w) = p$ then

$$\hat{\delta}_N(q_0, w) = \{p\}$$

We leave the proof to the reader. As a consequence, w is accepted by D if and only if it is accepted by N ; i.e., $L(D) = L(N)$. $\square$

A Bad Case for Subset Construction

- For an n -state NFA it is possible to build a DFA with 2^n states using subset construction.

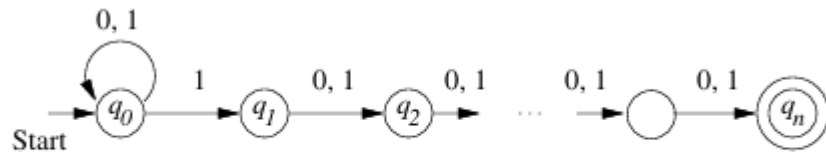


Figure 2.15: This NFA has no equivalent DFA with fewer than 2^n states

Example 2.13 : Consider the NFA N of Fig. 2.15. $L(N)$ is the set of all strings of 0's and 1's such that the n th symbol from the end is 1. Intuitively, a DFA D that accepts this language must remember the last n symbols it has read. Since any of 2^n subsets of the last n symbols could have been 1, if D has fewer than 2^n states, then there would be some state q such that D can be in state q after reading two different sequences of n bits, say $a_1a_2 \cdots a_n$ and $b_1b_2 \cdots b_n$.

Since the sequences are different, they must differ in some position, say $a_i \neq b_i$. Suppose (by symmetry) that $a_i = 1$ and $b_i = 0$. If $i = 1$, then q must be both an accepting state and a nonaccepting state, since $a_1a_2 \cdots a_n$ is accepted (the n th symbol from the end is 1) and $b_1b_2 \cdots b_n$ is not. If $i > 1$, then consider the state p that D enters after reading $i - 1$ 0's. Then p must be both accepting and nonaccepting, since $a_ia_{i+1} \cdots a_n00 \cdots 0$ is accepted and $b_ib_{i+1} \cdots b_n00 \cdots 0$ is not.

The Pigeonhole Principle

- If you have more pigeons than pigeon holes and each pigeon flies into some pigeonhole then there must be at least one hole that has more than one pigeon.
- In our example the pigeons are the sequences of n bits and the pigeonholes are the states. Since there are fewer states than sequences, one state must be assigned two sequences.
- It works for finite state automata, but does not apply to automata with infinite states.

To see why the finiteness of the number of pigeonholes is essential, consider the infinite situation where the pigeonholes correspond to integers $1, 2, \dots$. Number the pigeons $0, 1, 2, \dots$, so there is one more pigeon than there are pigeonholes. However, we can send pigeon i to hole $i + 1$ for all $i \geq 0$. Then each of the infinite number of pigeons gets a pigeonhole, and no two pigeons have to share a pigeonhole.

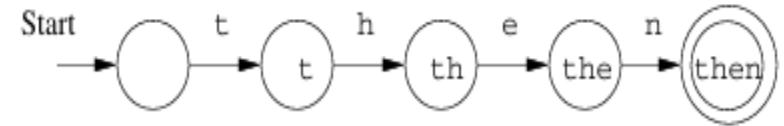
Dead State and DFA's Missing Transitions

Dead States and DFA's Missing Some Transitions

We have formally defined a DFA to have a transition from any state, on any input symbol, to exactly one state. However, sometimes, it is more convenient to design the DFA to “die” in situations where we know it is impossible for any extension of the input sequence to be accepted. For instance, observe the automaton of Fig. 1.2, which did its job by recognizing a single keyword, **then**, and nothing else. Technically, this automaton is not a DFA, because it lacks transitions on most symbols from each of its states.

However, such an automaton is an NFA. If we use the subset construction to convert it to a DFA, the automaton looks almost the same, but it includes a *dead state*, that is, a nonaccepting state that goes to itself on every possible input symbol. The dead state corresponds to $\emptyset$, the empty set of states of the automaton of Fig. 1.2.

In general, we can add a dead state to any automaton that has *no more* than one transition for any state and input symbol. Then, add a transition to the dead state from each other state q , on all input symbols for which q has no other transition. The result will be a DFA in the strict sense. Thus, we shall sometimes refer to an automaton as a DFA if it has *at most* one transition out of any state on any symbol, rather than if it has *exactly one* transition.



Exercise 2.3.6--think

- Section 2.3 of Chapter 2.